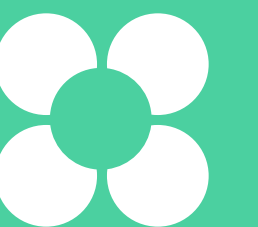


Spring boot: назначение, внутреннее устройство



План занятия

- 1 Spring boot
- 2 Конфигурация приложения

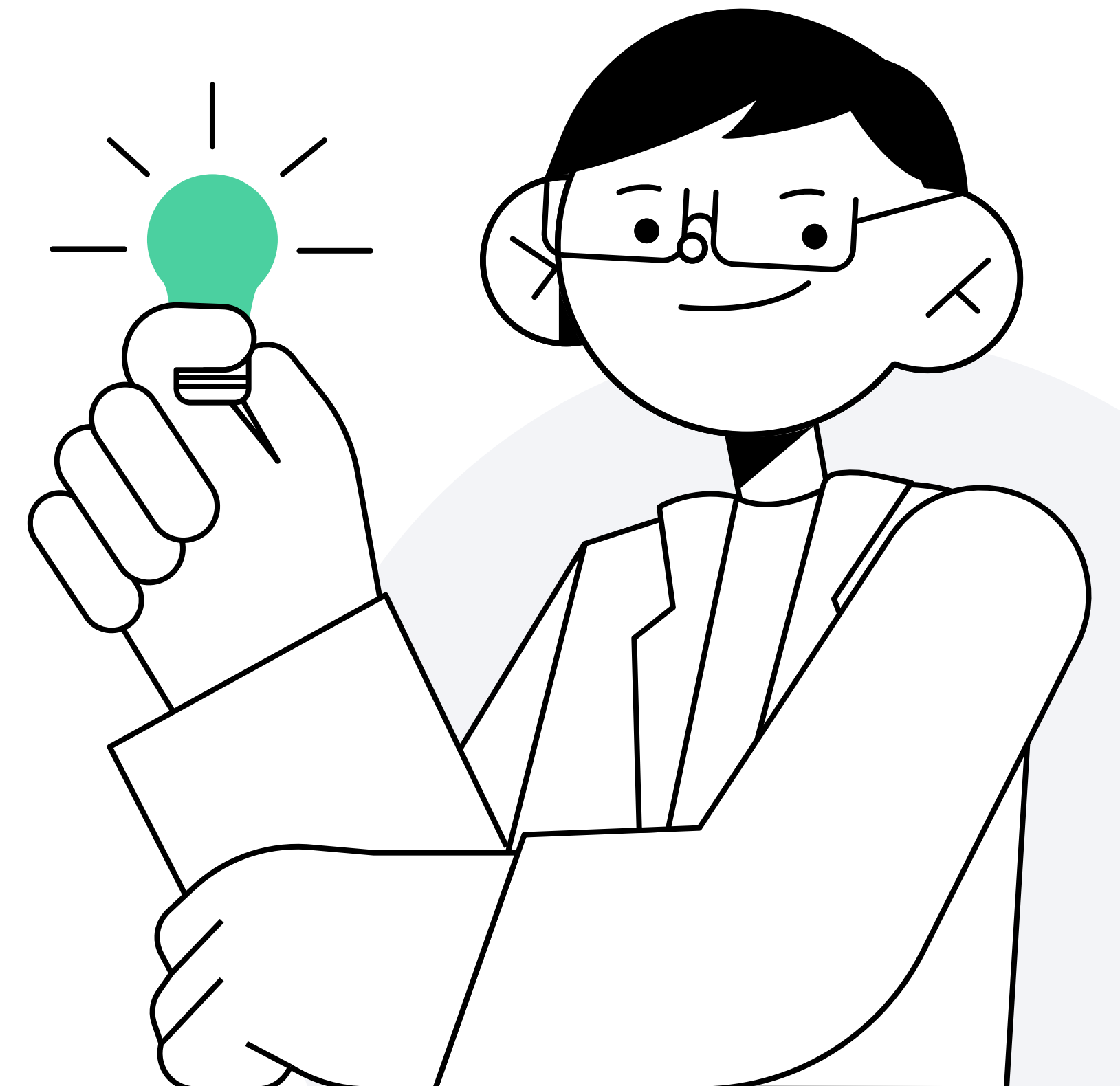


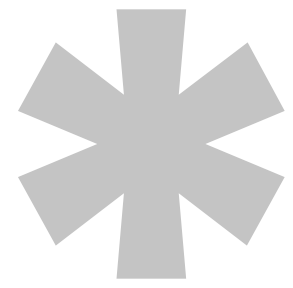
Spring boot

Василий Дорохин
QA-engineer Quadcode

Тема занятия

- 1 Преимущества использования Spring boot
- 2 Создание первого Spring Boot-приложения





Spring boot — это современный framework для создания Java-приложений, который отличается высоким уровнем абстракции и предоставляет быстрый способ создания production-ready приложений



Ключевые достоинства Spring boot

К ключевым особенностям Spring boot можно отнести

- Stand-alone приложения
- Embedded containers
- Автоконфигурации
- 'Starter' dependencies
- Metrics



@SpringBootApplication

Функционал, включаемый аннотацией @SpringBootApplication по умолчанию

→ Конфигурирование

Класс может содержать декларацию бинов с помощью @Bean

→ Автоконфигурации

Включает функционал автоконфигурирования @EnableAutoConfiguration

→ Поиск бинов и конфигураций

С текущего пакета начнётся поиск классов, помеченных аннотациями @Component или @Configuration

@SpringBootApplication

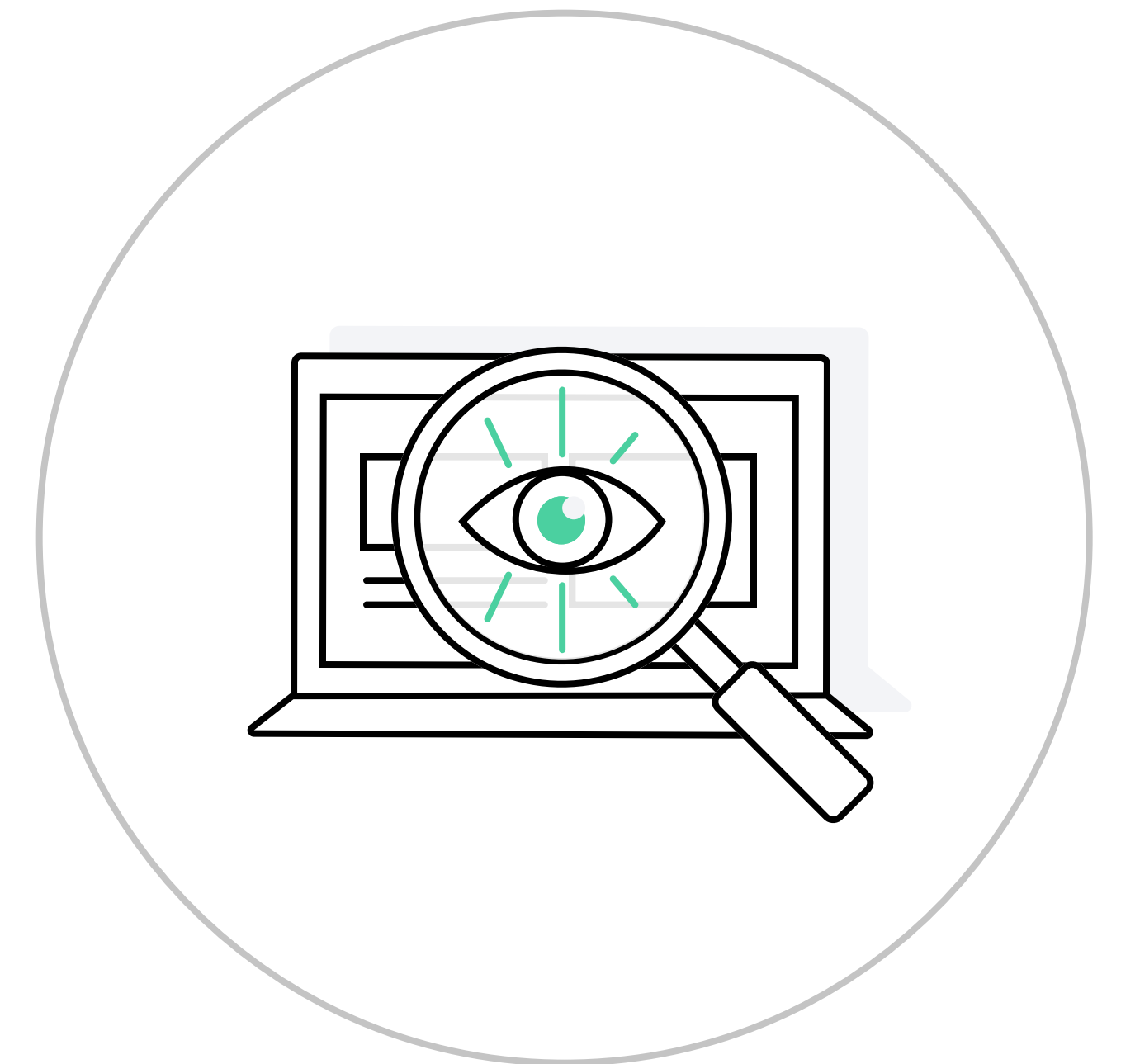
Функционал, настраиваемый в аннотации @SpringBootApplication:

- выключение автоконфигураций через указание класса или через указание полного название класса
- указание пакета, с которого следует начать поиск классов, аннотированных @Component или @Configuration

Endpoint

Actuator предоставляет следующие модули для мониторинга:

- **configprops** — показывает весь список параметров приложения
- **health** — показывает состояние приложения
- **metrics** — показывает специальные метрики приложения
- **другие**

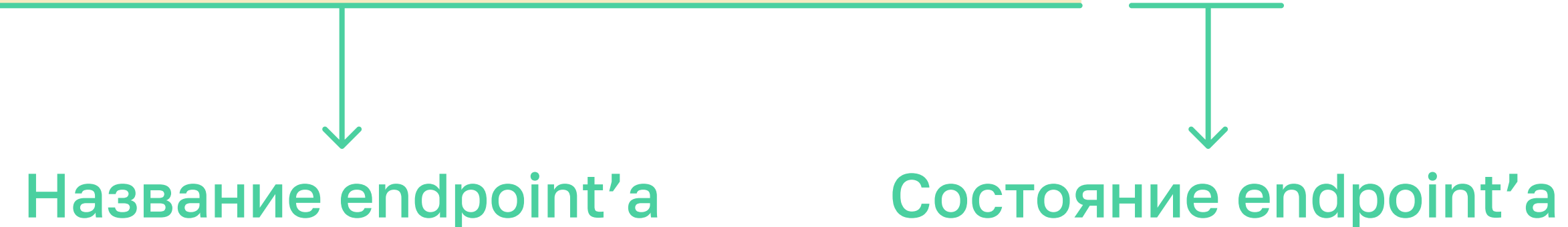


Включение Endpoint

По умолчанию большинство метрик выключены

Включение/выключение какого-либо endpoint производится проставлением значения параметра в application.properties:

```
management.endpoint.shutdown.enabled=true
```



Название endpoint'а

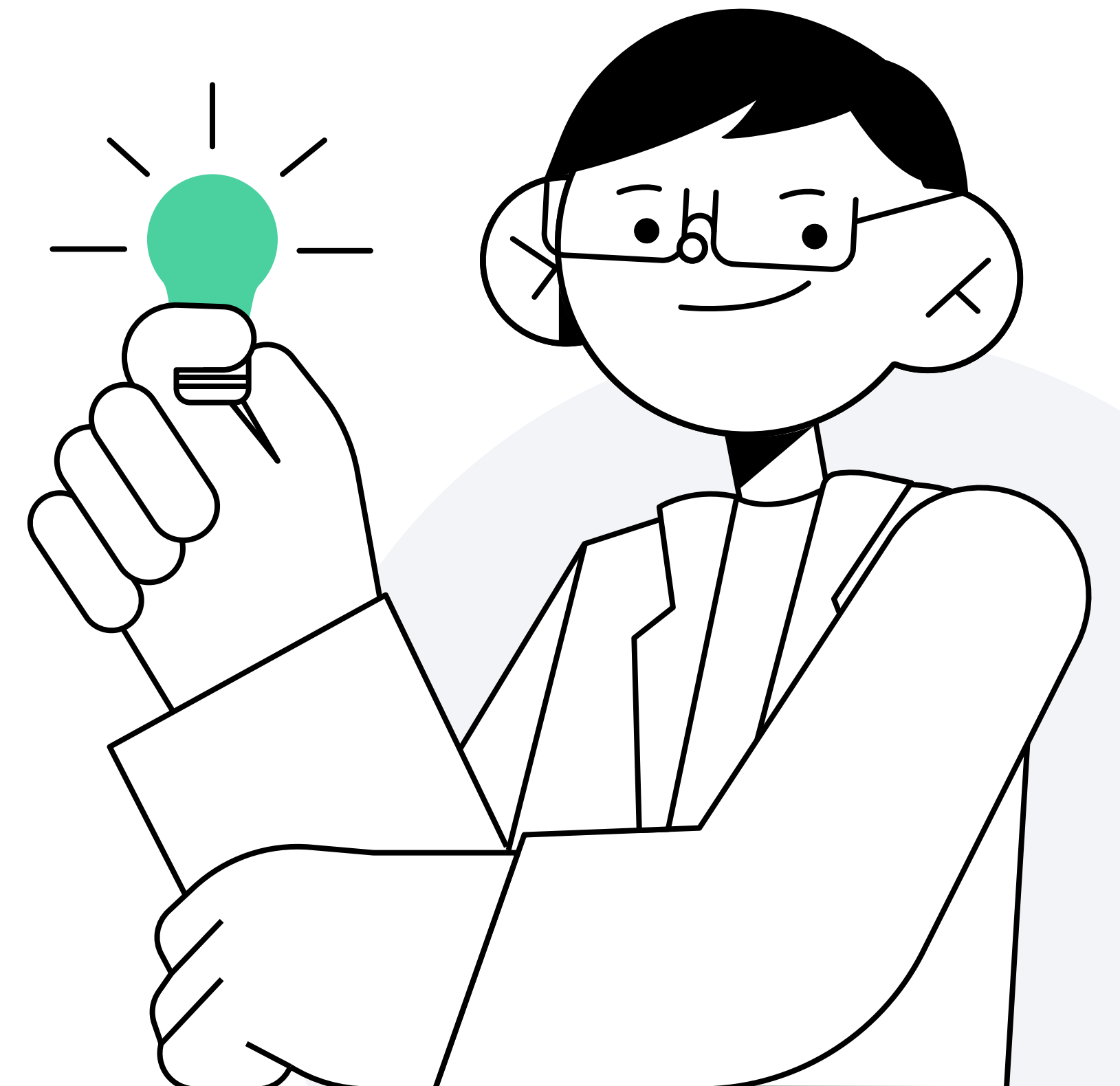
Состояние endpoint'а

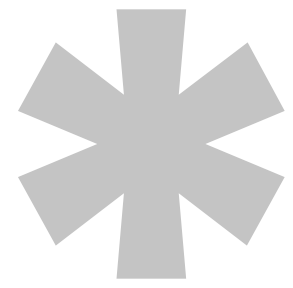
Конфигурация приложения

Василий Дорохин
QA-engineer Quadcode

Тема занятия

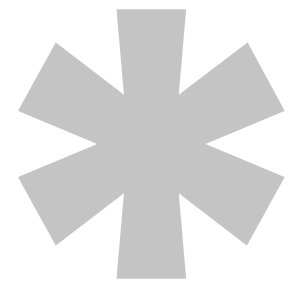
- 1 Автоконфигурации
- 2 @ConfigurationProperties и @Value





Автоконфигурации — это механизм, который конфигурирует и добавляет бины в `ApplicationContext` на основе зависимостей, которые есть в `classpath` приложения

Включается аннотацией `@EnableAutoConfiguration`



@Conditional

Аннотация Conditional — это механизм, который используется для включения в контекст приложения необходимых бинов в зависимости от выполнения условий, заданных в классе, реализующем интерфейс Condition

@Conditional

```
@Conditional(ProfileCondition.class)
public @interface Profile {

    class ProfileCondition implements Condition {

        @Override
        public boolean matches(ConditionContext context, AnnotatedTypeMetadata metadata) {
            MultiValueMap<String, Object> attrs = metadata.getAllAnnotationAttributes(Profile.class.getName());
            if (attrs != null) {
                for (Object value : attrs.get("value")) {
                    if (context.getEnvironment().acceptsProfiles((String[]) value)) {
                        return true;
                    }
                }
                return false;
            }
            return true;
        }
    }
}
```

Виды @Conditional

@ConditionalOnBean

регистрация бина в контексте в случае, если в контексте уже присутствует бин с заданным в аннотации названием

@ConditionalOnMissingBean

регистрация бина в контексте в случае, если в контексте ещё не зарегистрирован бин с заданным в аннотации названием

@ConditionalOnClass

регистрация бина в контексте в случае, если в classpath есть класс, заданный в аннотации

@ConditionalOnProperty

регистрация бина в контексте в случае, если задан определённый параметр в application.properties

Порядок поиска

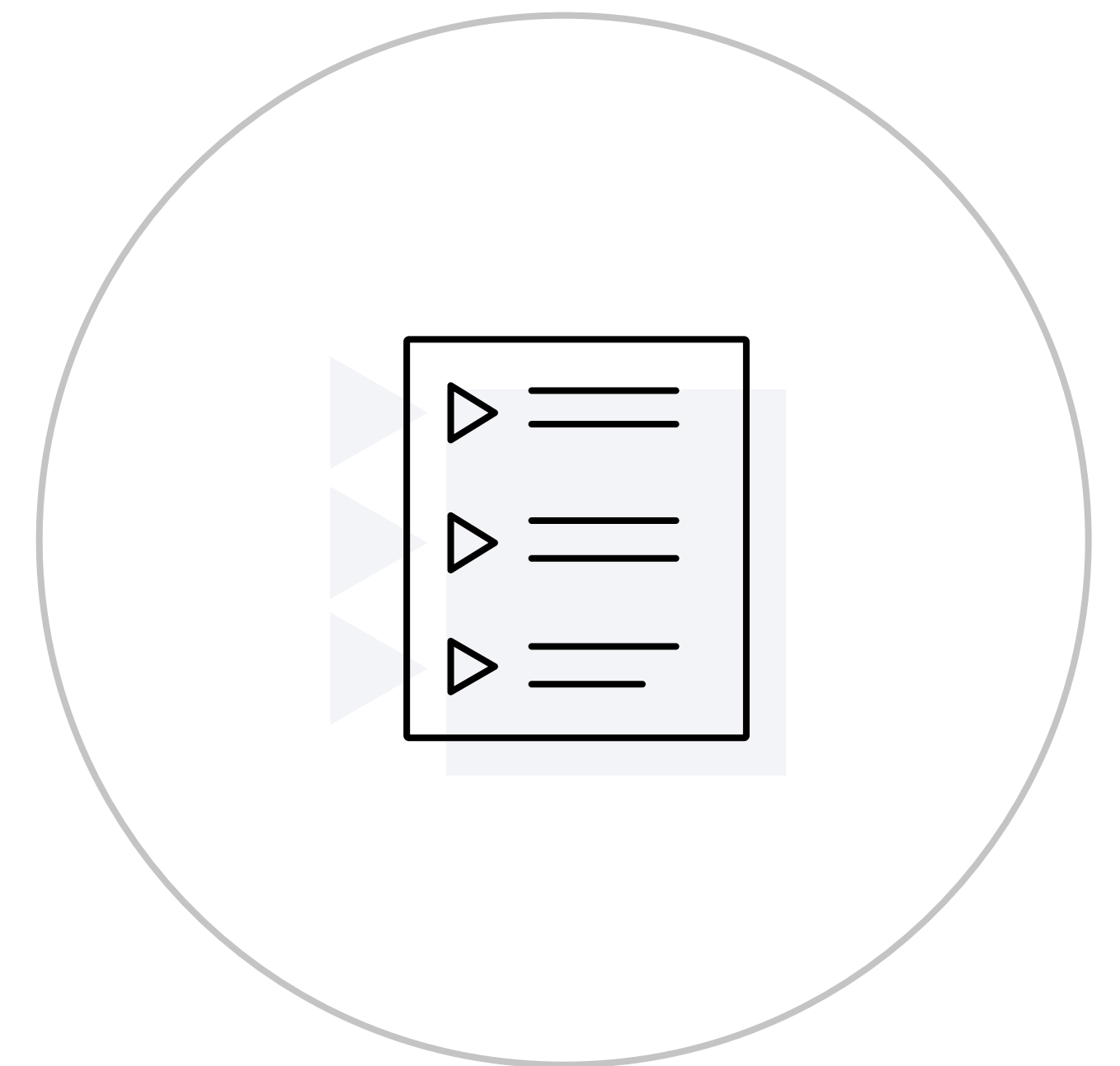
Порядок поиска application.(properties|yaml) по директориям:

- корень classpath
- ↓
- папка /config в classpath
- ↓
- текущая директория
- ↓
- папка /config в текущей директории
- ↓
- дочерние директории поддиректории /config

Relaxed binding

Relaxed binding позволяет использовать параметры в различных форматах:

- ✓ kebab case: `my-property.id` (рекомендован)
- ✓ camel case: `myProperty.id`
- ✓ snake case: `my_property.id`
- ✓ upper underscore: `MY_PROPERTY.ID`



@Value

@Value позволяет внедрять пользовательские параметры в поля:

- ✓ подставляет значения по полному совпадению placeholder
- ⋮ через двоеточие можно указать дефолтное значение, если этот параметр не будет найден:

```
public class MyProperties {  
  
    @Value("my.properties.enabled:true")  
    private boolean enabled;  
  
    @Value("my.properties.id:1111")  
    private int id;  
  
    @Value("my.properties.name")  
    private String name;  
  
    public boolean isEnabled() { return enabled; }  
  
}
```


@ConfigurationProperties vs @Value

Сравнения функциональности:

Фича	@ConfigurationProperties	@Value
Relaxed binding	Да	Ограниченно
Использование SpEL	Нет	Да

Итоги

- Spring boot является одним из самых удобных фреймворков для прототипирования и сборки production-ready приложений. Также, он заботится о том, чтобы вы не писали каждый раз один и те же бины, и могли все это конфигурировать в одном файле.